Tecnológico de Monterrey

# 2. Database Modeling

## Exercise for the Challenge

Software Construction and Decision Making

Grupo 501


| | |
|---|---|
| Daniela Janet Gil Gonzalez | A01752908 |
| Yuhao Liu | A01787782 |
| Mariana Ramirez Cervera | A01787819 |


28 de abril del 2026

The proposed relational schema was redesigned to better represent the dynamic information generated by the videogame. Since the videogame is a roguelike deck-building game, the database needs to store repeated gameplay attempts, player progress, combat events, card usage, math problem performance, and saved skill cards. The second version of the diagram focuses less on static data and more on information that is useful for gameplay functionality and statistics.

Some elements from the first version were removed because they did not provide useful database value. Other tables were adjusted to better support dynamic data, such as runs, problem attempts, cards used, and persistent skill cards.

1. **Player Table:**

This table is necessary because the game needs to identify each player and connect that player with their runs, progress, decks, skill cards, and statistics. Each player can have multiple runs and multiple gameplay records over time.

2. **Run Table:**

This table is one of the most important dynamic tables in the schema because a new record is created every time the player starts a new run. It stores the player who played, the level attempted, the duration of the run, the final result, and the number of enemies defeated. This helps for the overall player performance statistics, since it answers questions such as How many runs has the player completed? How often does the player win or lose? Which level is the hardest? How many enemies does the player usually defeat per run? How long does an average run last?

3. **Player Stats Table:**

This table summarizes the player's long-term progress. It is useful because the game may need to quickly display the player's current progress, total play time, number of solved problems, enemies defeated, and average response speed. Some of this information could also be calculated from other tables, such as Run and Problem Stats. However, this table is useful as a summary table because it allows the game to load player progress more efficiently.

4. **Enemy Table:**

This table is mostly a catalog table, but it is still necessary because combat records need to reference which enemy was fought. Each enemy has a name, type, world level, health points, attack power, and possibly a skill. This table helps the game mechanics manage enemy difficulty and behavior. It also helps statistics related to enemy performance. Also the database can answer questions such as Which enemy type is most difficult? Which enemies defeat players most often? How much damage do certain enemies usually cause? Which world level contains stronger enemies?

5. **Card Table:**

Cards are one of the main mechanics of the game, so this table is necessary. It defines the available cards, their type, their world level, their description, their power value, and their effect. This table is useful because it connects to deck selection, skill cards, and cards used during combat. It can also be used for statistics such as Which cards are used most often? Which cards are related to winning runs? Which card effects are most useful? Which card types are most common in combat?

## 6. Deck Table:

This table allows the database to know which cards are associated with a player's deck. Since deck-building is part of the game, the table supports the connection between the player and the cards they choose. For a more dynamic design, the deck could also be connected directly to the Run table. This would allow the database to know exactly which cards were selected for each run, which is useful for analyzing if certain cards are related to better results.

## 7. Skill Deck Table:

This table is important because the game has roguelike mechanics. This means that many elements can reset after a loss, but some progress remains saved. In this case, the saved progress is unlocked skill cards. The Skill Deck table is different from the regular deck because it represents player progress that remains even after the player loses the run. It stores which skill cards each player has unlocked permanently. This table allows the database to answer questions such as: Which skill cards has the player unlocked? How much persistent progress does the player have? Which skill cards are unlocked most often? Do unlocked skill cards improve run results?

## 8. Problem Stats Table:

This table is very important because math problems are one of the main mechanics of the game. It stores the problem expression, the difficulty, the operation type, the response time, and the answer record. This is dynamic data because each record is created when the player receives and answers a math problem during a run. This table supports statistics such as Which operation type is hardest? How fast does the player answer? Does the player improve over time? Which difficulty level causes more mistakes? How many problems does the player solve per run?

## 9. Combat Table:

This table connects the run, enemy, and problem involved in a combat event. It records the timer result, damage dealt, combat result, and the date or time when the combat happened. Combat is one of the most important gameplay interactions because the player must use cards and solve math problems to defeat enemies. This table allows the database to analyze the relationship between player performance, math results, enemies, and combat outcomes. It is useful for questions such as How many combats happen per run? Which enemies are most difficult? How much damage does the player usually deal? Does faster response time improve combat results? Which combats are usually lost?

### 10. Cards Used Table:

This table is dynamic because a new record can be created whenever a player uses a card in combat. It connects the Combat table with the Card table. This table is useful because it records actual gameplay behavior, not just the cards that exist in the game. It helps analyze the effectiveness of different cards. The database can answer questions such as Which cards are used most often? Which cards are used in winning combats? Which cards produce better results? Which card effects are most useful?

## Normalization:

First Normal Form: The diagram follows the First Normal Form because each table stores atomic values. For example, each row in Card represents one card, and each row in Combat represents one combat event.

Second Normal Form: The diagram follows the Second Normal Form because the attributes in each table depend on the primary key of that table. For example, in the Combat table, attributes such as damage_dealt, timer_result, and combat_result depend on combatID.

Third Normal Form: The diagram supports Third Normal Form because information is separated into independent entities and repeated data is avoided. For example, enemy information is stored only in the Enemy table instead of being repeated in every combat record. Card information is stored only in the Card table instead of being repeated every time a card is used.

This organization reduces redundancy and makes the database easier to maintain.

## Conclusion

This relational scheme better supports the main logic of Math Smash: Card Adventure by focusing on dynamic gameplay information. It separates information into clear tables, avoids repetition, and connects the most important gameplay elements through relationships. It stores player accounts, runs, progress, enemies, cards, decks, skill cards, math problem performance, combat events, and cards used during combat. This makes the database useful both for gameplay functionality and for future analysis of player performance.